

Building InterMine databases from RDF Portal

Russell Y. Neches ¹, Shuichi Kawashima ², Toshiaki Katayama ², and Gos Micklem ^{3, 2}

BioHackathon series:
DBCLS BioHackathon 2026
Matsuyama, Japan
[rdfc2im](#)

¹ Institute for Chemical Research, Kyoto University, Kyoto, Japan  ² National Institute of Genetics, Chiba, Japan  ³ Department of Genetics, University of Cambridge, Cambridge, UK 

Submitted: 19 Sep 2026

License:
Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

Abstract

InterMine databases such as HumanMine integrate many biological data sources, but each source needs its own loader, and keeping those loaders current is costly. The public HumanMine has not had a data release since February 2022. RDF Portal, operated by the Database Division for Life Science, now serves many of the same sources as reviewed RDF, and describes each dataset with an rdf-config model. At the DBCLS BioHackathon 2026 we developed [rdfc2im](#), a tool that maps rdf-config models onto the InterMine data model, keeps the mapping as reviewable data with the evidence for every row, uses RDF Config to generate SPARQL queries to acquire data from RDF Portal, and writes the data into a form that InterMine's standard loader integrates without new code. We used it to build a working demonstration HumanMine from nine sources and a panel of 113 food- and drug-metabolism genes, with NCBI Gene, the Gene Ontology and Reactome loaded in full. The mine supports search, template queries and list analysis. Real loads found problems that static checks did not, most of them related to the way objects from different sources are identified and merged, and to the limits of public SPARQL endpoints. This report describes our proof of concept. We list what remains to be done before [rdfc2im](#) can rebuild a full HumanMine from each RDF Portal release.

Table 1: Glossary. Terms used in this report, as they are used here.

Term	Meaning
RDF, SPARQL	The W3C standards for data as subject-predicate-object triples, and for querying it. A <i>SPARQL endpoint</i> is a web service that answers SPARQL queries.
RDF Portal	The DBCLS service that hosts reviewed life science RDF datasets behind public SPARQL endpoints.
rdf-config model	A YAML description of one RDF dataset: its classes (<i>subjects</i>), the predicates that connect them, how many values each may have, and example values.
InterMine, mine	A data warehouse system for biological data, and one database built with it (here, HumanMine).
Source	One input to a mine, loaded by its own loader. A <i>stock</i> source is one that HumanMine already runs.
Converter	The Java code in a stock source that turns a provider's files into InterMine objects.
Items XML	InterMine's generic input format: a file of objects (<i>items</i>) with their attributes and references.
Integration key	The fields InterMine uses to decide that objects from two sources are the same object, for example a gene's NCBI Gene identifier.

Term	Meaning
Priorities	The configuration that decides which source's value wins when two sources give one field different values.
Additions	A source's extension to the core InterMine data model: the classes and fields it adds.
SSSOM	A standard tab-separated format for mappings between vocabularies (Matentzoglou et al., 2022).
BlueGenes	InterMine's current web interface.

Introduction

Integrative analysis in biology depends on bringing independent databases together, so that the evidence can be queried in one place. Evidence might pertain to genes, proteins, organisms, diseases, or ecological relationships. InterMine is an open-source data warehouse system built for this purpose (Smith et al., 2012). From one object model, InterMine generates a web interface, a query builder, a template search capability, list analysis with enrichment statistics, and a REST API with client libraries in several commonly used languages (Kalderimis et al., 2014). HumanMine is the InterMine instance for human data. Its 2022 release integrated over 60 datasets from over 30 sources, among them NCBI Gene, HGNC, UniProt, the Gene Ontology, Reactome, ClinVar and the GWAS Catalog, into one searchable database (Lyne et al., 2022).

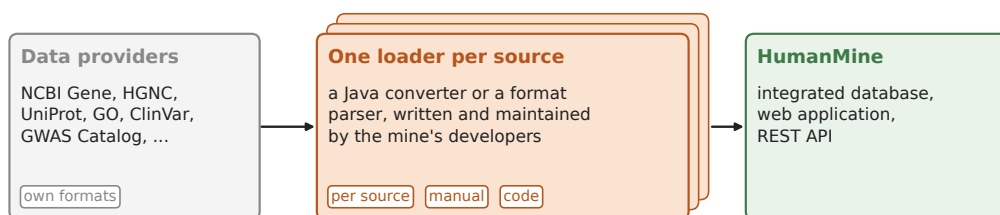
A key cost of a mine is in keeping its data current. Each source reaches the warehouse through its own loader: a parser for a standard format, a custom Java converter, or a file in InterMine's own XML item format (Smith et al., 2012). When a provider changes its release format, its loader must change too, and a mine with dozens of sources carries dozens of such dependencies. The cost of this maintenance is one reason that the public HumanMine still reports its release as 12 February 2022.

Many of the same sources are now available from one service, in one format. RDF Portal, operated by the Database Division for Life Science (DBCLS), hosts life science datasets in RDF behind public SPARQL endpoints (Database Center for Life Science, 2026a). It began as the NBDC RDF portal, which reviewed each submitted dataset against guidelines for interoperability: typed primary resources, human-readable labels, local identifiers in `dcterms:identifier`, and identifiers.org URIs for cross-references (Kawashima et al., 2018). In 2018 the portal held 21 datasets from Japanese groups, with 45.5 billion triples (Kawashima et al., 2018). In September 2026 it lists 64 datasets with 229.2 billion triples, including UniProt, NCBI Gene, HGNC, Ensembl, ClinVar, Reactome and PubMed (Database Center for Life Science, 2026a). Each dataset on the portal also has an `rdf-config` model (Database Center for Life Science, 2026b): a short YAML description of the classes in the dataset, the properties that connect them, and the form of their values. The portal uses these models to draw schema diagrams, to generate SPARQL, and to configure its GraphQL and AI-agent interfaces.

Together, these two resources suggest a different way to build a mine. If the structure of each source is already described in a machine-readable model, then the mapping from that model to the InterMine model can be written once, as data and not as code. One generic tool can then fetch, transform and load any source that has such a model (Figure 1). `rdc2im` (`rdf-config` to InterMine) is that tool. It aligns `rdf-config` models to the HumanMine data model and records each alignment in a mapping file that a curator can review. From the mapping it uses `rdf-config` to generate SPARQL queries, fetches the results, and writes InterMine Items XML for the stock InterMine loader. In this report we describe `rdc2im` and its use at the DBCLS BioHackathon 2026, where we built a working demonstration HumanMine for a panel of 113 food- and drug-metabolism genes from nine RDF sources, most of them on RDF Portal. We also record the problems that only a real InterMine load can reveal, which static checks of the

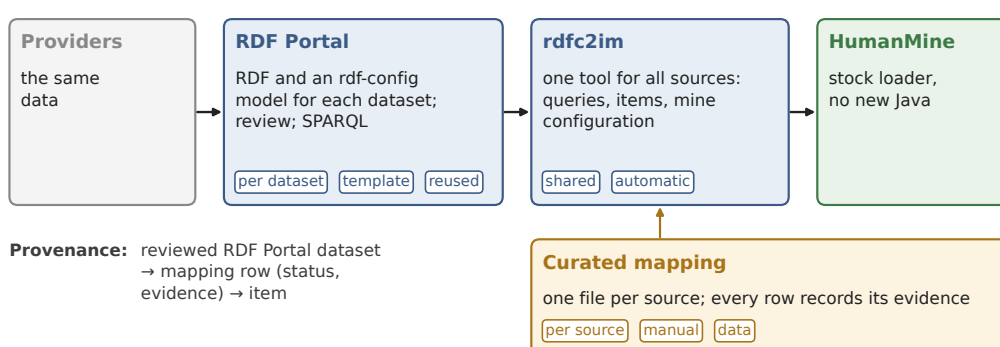
mapping did not.

(a) HumanMine today



Provenance: a DataSet for each loader; how each value was derived is written in that loader's Java code

(b) With RDF Portal and rdfe2im



Provenance: reviewed RDF Portal dataset
→ mapping row (status, evidence) → item

Figure 1: How a mine gets its data. (a) Today each HumanMine source has its own loader, written and maintained as code by the mine's developers. (b) With RDF Portal and rdfe2im, the work for each dataset moves upstream: RDF Portal holds each dataset's RDF and an rdf-config model written in rdf-config's common form, made once and reused by every consumer. rdfe2im is one tool for all sources, and what remains for each source is a curated mapping, kept as data with the evidence for every row. Tags show scope (*per source*, *per dataset*, *shared*, *reused*), effort (*manual*, *automatic*) and form (*code*, *template*, *data*).

Approach

rdfe2im is a command-line tool written in Python. It follows the four steps of the workflow set out at the previous DBCLS hackathon (Figure 2): prepare the two data models, map one onto the other, extract the data with SPARQL, and load it into InterMine. Each step writes plain text files that a person can read. Under version control there are only inputs and curation decisions; everything else is regenerated.

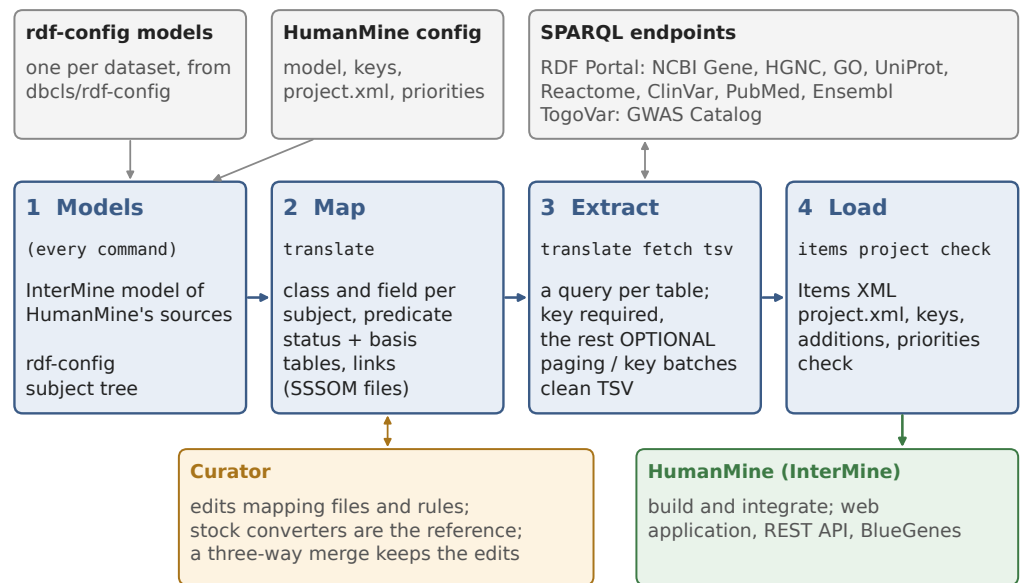


Figure 2: The `rdfc2im` pipeline. The four steps follow the workflow proposed at the previous DBCLS hackathon. Arrows show which step reads each input; the commands that run each step are in monospace. The curator edits the mapping files between runs, and a three-way merge keeps those edits when the mapping is regenerated.

Inputs

`rdfc2im` reads three kinds of input. The first is the `rdf-config` model of each source, taken from the `rdf-config` repository (Database Center for Life Science, 2026b): the subjects and predicates of the dataset, the cardinality of each predicate, example values, prefixes, and the SPARQL endpoint. The second is HumanMine's own configuration: the InterMine core model, the additions and integration keys of each source that HumanMine runs, its source list (`project.xml`), and its data source priorities, taken from the InterMine and HumanMine repositories. `rdfc2im` reasons over the InterMine model limited to HumanMine's sources, plus a short list of model extensions that the project approved (for example, `Gene.typeOfGene` and `Pathway.description`). The third is the data itself, fetched from the endpoints that the `rdf-config` models name. In the demonstration build, every source came from RDF Portal except the GWAS Catalog, which came from TogoVar. This includes UniProt: its queries are sent to RDF Portal's SIB mirror (`rdportal.org/sib/sparql`), not to UniProt's own `sparql.uniprot.org`, which appears only as a named graph inside that RDF Portal dataset.

Mapping

For each source, `rdfc2im` walks the `rdf-config` model from a root subject, for example the `Gene` of NCBI Gene. It binds each subject to an InterMine class, and each predicate to a field of that class. The first rule that applies decides:

1. an explicit rule for this source
2. a match between the RDF type or predicate IRI and the ontology term that the InterMine model gives a class or field
3. a general rule for that RDF type or predicate
4. a match between names
5. otherwise the row is tagged as "todo" and left for a curator.

Every column that `rdfc2im` extracts is an attribute of one object. When a value belongs to a different object, for example a synonym of a gene, the mapping names the reference that

connects the two (`Gene.synonyms`). Objects are linked only where a mapping names the link.

The mapping also decides how the data extracted from RDF portal is split into tables. Each table is one SPARQL query and one file. The main table holds the root's identifier and its single-valued attributes. Each multi-valued predicate that creates other objects has a table of its own, holding the root's identifier and that one column, so that two multi-valued predicates never multiply each other's rows. For NCBI Gene this gives five tables: the gene itself, its synonyms, its alternative names, its Ensembl identifiers and its cross-references.

Curation

Each source has two mapping files that a curator can edit: one row per subject, and one row per predicate. The predicate file uses the SSSOM layout (Matentzoglou et al., 2022), with `rdcf2im`'s own columns declared as extensions. Every row has a *status* and a *basis*. The status decides whether the row loads: *sure* (backed by evidence), *guess* (a proposal to review) and *human* (the curator's own edit) are all loaded; *todo* (undecided) and *drop* are not loaded, with the reason given. The basis records where the mapping came from, for example a term match or a named converter.

The reference for curation is the stock HumanMine converter for the same source. A field that the converter writes is mapped, and its basis names the converter. A field that the converter never writes, or that the model cannot hold, is dropped with a reason. A field beyond what the converter writes loads only as a guess. A choice that the converter does not settle stays *todo*. Rules that apply across runs are kept in one reviewable file, with their evidence. When `rdcf2im` regenerates the mapping files, a three-way merge against its previous automatic output ensures that every value that a curator changed is preserved.

Using a standard mapping format pays off beyond loading: `rdcf2im` can project the SSSOM mapping back onto each source's own `rdf-config` model, producing a copy of that source's `model.yaml` with every `rdf-config` variable relabelled to the InterMine field it maps onto, in the same syntax `rdf-config` itself uses. This gives a curator, or an `rdf-config` maintainer, a one-file view of the whole mapping in the vocabulary they already read the source in, including which predicates are still unmapped and why.

Extraction

`rdcf2im` generates one SPARQL query per table. Only the root's identifier is a required pattern. Everything else is `OPTIONAL`, so a record that lacks one value still loads. A constraint on a required pattern limits a source, for example to human genes:

```
SELECT DISTINCT ?id ?taxid ?gene_synonym
FROM <http://rdfportal.org/dataset/ncbigene>
WHERE {
  ?Gene a insdc:Gene .
  ?Gene dct:identifier ?id .
  ?Gene ncbio:taxid ?taxid .
  VALUES ?taxid { taxid:9606 }
  OPTIONAL { ?Gene insdc:gene_synonym ?gene_synonym . }
```

The fetch step pages through each table in a fixed order. SPARQL endpoints limit how many sorted rows one query may page through: 200,000 on RDF Portal and 10,000 on TogoVar. A table larger than the limit is fetched instead in batches of 1,000 root identifiers, listed in a `VALUES` clause. A table that still cannot be fetched in full is reported as incomplete, never as complete.

The cleaning step turns RDF terms into plain values. For each column it applies a filter to the raw value, then a chain of transforms, for example taking the local name of an IRI. A row

whose identifier is empty, or fails its filter, is dropped. Any other value that fails its filter is left empty.

Loading

rdfc2im writes one Items XML file per source. We chose Items XML over InterMine's loader for tab-delimited files because it states every reference explicitly, so objects are linked exactly as defined by the mapping. The files are loaded by InterMine's standard Items XML loader, registered as a source type of its own so that rdfc2im's generated keys and model additions travel with it. No new Java code is needed.

Within a source, multiple rows that describe a single object become one item: rdfc2im identifies an object by the first integration key of its class that the row fills. So a gene met in five tables is one item. Values for numeric fields are checked against the field's type, and a value that does not fit (such as "NR", for "not reported") is dropped and reported.

rdfc2im also generates the configuration that the mine needs. Each rdfc2im source has one of three roles. It *replaces* a stock source (NCBI Gene, HGNC, GO, HPO, MP, ClinVar and the GWAS Catalog). It loads *alongside* a stock source, right after it, filling only what the stock source leaves empty (UniProt, Reactome). Or it *adds* data that HumanMine did not have (for example Ensembl and PubMed). The generated additions include every class the mappings use that is not in InterMine's core model, because removing a stock source also removes the classes it declared. The generated source priorities are HumanMine's own, with each replacing source in the place of the source it replaces. Final checks can fail on an unknown field, a link whose type does not fit, two columns that write one field of one object, or data that a replaced stock source loaded and its replacement does not, unless that gap is accepted with a reason.

Scope and the demonstration build

A build can be limited to one or more species and to a list of genes without editing the mapping: the limit is applied to a copy of the mapping when the queries are generated. Each source names the field that identifies its genes, and a live lookup translates gene symbols into that source's own identifiers (NCBI Gene or Ensembl). Where the source data breaks an integration key that HumanMine assumes, a configured rule decides which object keeps the value. If the rule cannot decide, no object keeps it. For example, 255 Ensembl identifiers are each claimed by two or more genes in NCBI Gene; the identifier stays only on a gene that is the one protein-coding claimant.

The demonstration build used a panel of 113 human genes involved in food and drug metabolism, in seven groups from drug-metabolizing enzymes to taste and appetite receptors. NCBI Gene and Reactome were loaded in full. HGNC, Ensembl, UniProt, ClinVar and the GWAS Catalog were limited to the panel, and PubMed to the publications that the other sources cite. The mine was an InterMine 5 build of HumanMine, served with its database, web application and BlueGenes in containers on one machine.

Results

The demonstration mine

We built a working HumanMine from nine sources (Table 2). Every source loaded into one mine, and objects from different sources merged as intended: for example, BRCA1 and TP53 are each one Gene that carries identifiers and chromosome from NCBI Gene, the approved symbol and cytogenetic location from HGNC, and synonyms from both.

Table 2: The sources of the demonstration build. *Role* says how the source relates to HumanMine's own source for the same data (see Approach). *Scope* says how much was loaded. Counts for NCBI Gene, the Gene Ontology, Reactome, the GWAS Catalog and PubMed are from the load records in the repository (LOAD-TRIAL.md); counts for HGNC, Ensembl, UniProt and ClinVar were taken from each source's items file.

Source	Endpoint	Role	Scope	Loaded
NCBI Gene	RDF Portal	replaces	all human genes	193,288 genes
Gene Ontology	RDF Portal	replaces	all terms	48,340 terms
Reactome	RDF Portal	alongside	all human pathways	2,883 pathways
HGNC	RDF Portal	replaces	gene panel	113 genes
Ensembl	RDF Portal	adds	gene panel	112 genes
UniProt	RDF Portal	alongside	gene panel	1,501 proteins
ClinVar	RDF Portal	replaces	gene panel	53,105 alleles
GWAS Catalog	TogoVar	replaces	gene panel	23,201 results, 2,505 SNPs
PubMed	RDF Portal	adds	cited publications	3,879 publications

Ensembl loads 112 of the 113 panel genes: one, *GSTT1*, has no Ensembl identifier in NCBI Gene's own cross-reference data, a gap confirmed against NCBI's records rather than an artifact of this build (a known consequence of *GSTT1*'s common deletion polymorphism). It is otherwise in the mine through NCBI Gene, HGNC and ClinVar.

The mine supports HumanMine's usual ways in. Keyword search finds genes by partial symbol (a search for "cyp" returns the CYP family genes). Template queries work (Figure 3): three of HumanMine's own templates still apply to this smaller mine (two needed their default organism changed to human), and we added six for the panel, each tagged into HumanMine's own category filter (Genomics, Proteins, Literature, GWAS, Gene Ontology, Disease, Variant) so they appear alongside the originals rather than as an unsorted extra group. The BlueGenes report page for a panel gene shows data from every source on one page: for example, CYP2D6 carries its NCBI Gene and Ensembl identifiers, HGNC symbol and cytogenetic location, and UniProt proteins on one page, verified through the same REST query interface BlueGenes itself uses.

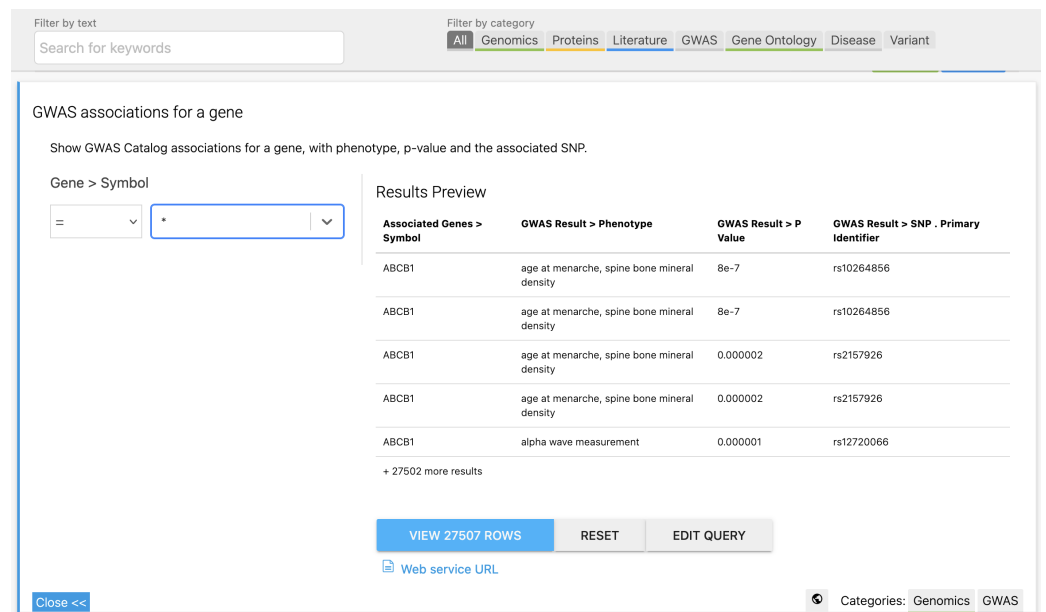


Figure 3: The “GWAS associations for a gene” template, one of the six added for the panel, with its gene constraint left open (*), so the preview lists associations for every gene in the mine. HumanMine’s category filter (top) shows the added templates tagged into the same categories as the three original templates.

Some features are empty, and each gap traces to data that was not loaded rather than to a fault in the interface. There are no genome coordinates, so the chromosome distribution for SNPs and HumanMine’s location-based features are empty. GO annotations, protein domains and interactions come from stock sources that this build did not run. The GWAS study enrichment widget fails, because GWASResult has no integration key in HumanMine: one association that names several genes becomes several rows (2,828 of 23,201 are duplicates).

Curation

The system has been set up to enable curation but due to time constraints our curation pass was not fully rigorous, and much of the information given depends on automated assignments. We were able to demonstrate that the curation workflow works correctly, and that it is often possible to validate automated decisions through their outcomes via independent sources of evidence. Across 15 translated sources, the active rows of the mapping files (those not under a skipped subject) are 124 sure, 64 guess, 38 todo and 309 drop. Of the 38 open rows, 21 are ClinVar genome coordinates, which are deferred. The shared rule file holds 449 rules with a recorded basis. 211 of them cite a stock HumanMine converter as evidence, and 18 rest on a match between an RDF term and an InterMine ontology term. So the stock converters, not term matching, supplied most of the mapping.

Integration test results

Each source’s first real load found problems that no earlier step had found, and the static checks passed before every one of them. Table 3 describes the issues. Most were about identity: which key an object merges on, and whether the objects it links to can merge too.

Table 3: Problems found by fetching and loading real data, grouped by kind. *Found* is the first stage that showed the problem: a full fetch, an InterMine load, or use of the running mine.

Kind	What happened	Found	Fix
Identity	An Ensembl gene's own IRI was guessed as its primary key, where HumanMine uses NCBI ids; every Ensembl gene became a duplicate	load	Drop the guessed key for that source
Identity	Ensembl rows carry only a gene's secondary key, but items were identified by the primary key alone, so one gene split into several items	load	Try each single-field key the row fills
Identity	Reactome pathways have no curated key; a multi-valued comment made rows differ, and 2,803 of 2,883 pathways loaded twice	use (the count was double the known number)	Fall back to a common identifier field
Identity	Genes reached from UniProt had no organism, so they could not merge: 113 duplicate genes. The same gap in the GWAS Catalog and ClinVar was fixed before loading	load	Attach the organism to the linked gene
Identity	255 Ensembl ids are each claimed by two or more genes in NCBI Gene, which breaks one of HumanMine's keys	load	Keep the id only where a rule picks one gene
Identity	ClinVar's rdf-config model identifies a record by its VCV accession; HumanMine keys alleles on the numeric AlleleID	reading the converter	Key on AlleleID; keep VCV as a secondary identifier
End-point	Past 200,000 sorted rows RDF Portal refuses the request; the truncated table was taken as complete (16.7% of gene synonyms lost)	full fetch	Detect the limit; fetch in key batches
End-point	Paging by comparing values returned wrong results on RDF Portal (14.7% of genes would have been lost)	full fetch	Not used; batch by listed keys instead
End-point	TogoVar has a 10,000-row limit and returns gene ids as IRIs, so gene-limited queries returned nothing	full fetch	Detect the limit during paging; match the end of the IRI
Data shape	OWL structure in OBO data: 16,178 blank-node parents and 11 OWL properties loaded as GO terms	full fetch, use	Filter on the raw value
Data shape	rdf-config's four example values for one PubMed predicate became four columns: 625 rows for a paper with 5 MeSH headings	fetch	Map the predicate once
Data shape	"NR" in a numeric GWAS field, and one non-ASCII character, each stopped a whole load	load	Drop values that do not parse; write ASCII
Mine	Removing a stock source also removed the model classes it declared (GOTerm) and data it alone loaded (Reactome's gene membership)	load, reading	Carry every non-core class; load alongside

Discussion

The demonstration shows that the approach works from end to end. A demonstration HumanMine can be built from RDF Portal data through the datasets' own rdf-config models, with the mapping kept as reviewable data and with no new Java code. It also demonstrates what RDF Portal's review guidelines and rdf-config models were designed to allow: a third party reused the data automatically, for a purpose its providers did not plan. The result is a proof of concept, not a replacement for HumanMine. It loads nine sources, most of them limited to a panel of 113 genes without genome coordinates, whereas HumanMine's 2022 release integrated about 60 datasets from around 30 data sources. We hope the lessons below help others who try the same with other mines or other RDF collections.

Term matching and stock converters

We expected the ontology terms in the InterMine model to drive most of the mapping, as in the workflow proposed at the previous hackathon. They supplied few rules (18 of 449). The stock HumanMine converters supplied far more (211), because a rebuilt mine must agree with the existing one on identifiers and on what each field means. Three early decisions changed when we checked them against a converter or the live mine: a predicate matched by name to `Allele.reference` held the reference sequence, not the reference base; ClinVar's rdf-config identifier differs from the key HumanMine uses; and a filter to reviewed UniProt entries would have removed about 83% of HumanMine's human proteins. For anyone replacing an established loader, the behavior of that loader is the specification. Recording the basis of every mapping row made these changes easy to find and to make.

What a translator needs from an rdf-config model

The rdf-config models were enough to generate every query in this work. Some things a translator needs are not stated in them, and had to come from elsewhere. A model does not mark which predicate identifies a record; `rdfc2im` takes this from InterMine's integration keys. Example values listed under one predicate may be alternative shapes of one value, not separate values (PubMed's MeSH headings). Blank nodes can carry structural labels that can be misinterpreted as data if not handled correctly (ClinVar's location labels). One predicate can mix different kinds of value: Reactome's comments hold both descriptions and curation notes, and Ensembl's `part_of` points at both a chromosome and an assembly. RDF Portal already uses its models to draw schema diagrams, to configure its GraphQL interface and to guide AI agents. Small additions, such as marking a record's identifier and whether examples are alternatives, would help every such consumer. We offer these as input to the development of rdf-config.

Identity is the hard part

Most problems in Table 3 were about identity. A generic loader must know each object's key, must make sure that every object that has to merge carries all of its key fields (a gene needs its organism as well as its identifier), and must detect where the source data breaks a key that the mine assumes. Where the data cannot be resolved cleanly, `rdfc2im` loads less rather than merging wrongly: it drops the row, or removes the ambiguous value. The internal checks did not catch the duplicated Reactome pathways; a comparison with the known number of human pathways did. Comparing counts against outside knowledge should be a routine step.

Replacing a pipeline piece by piece

Replacing a stock source is not the same as mapping its data. A source can load everything its RDF offers and still remove what the stock converter loaded from other files, such as Reactome's links between pathways and genes. The *alongside* role and the replacement check

exist for this reason. The check can only compare what a stock source declares, not what it loads, so it needs a recorded reason for each accepted gap. A second obstacle is HumanMine's own web configuration, which assumes that all of its sources are loaded. A mine built from a subset of sources, like our panel, has to trim four configuration files by hand. Small, focused mines would be easier to build if this were generated from the model the mine actually has.

Public SPARQL endpoints for bulk extraction

Endpoints differ in their limits and behavior, and the standard ways to page through results failed on them: a fixed limit on sorted results cut tables short without an error, and paging by comparing values returned wrong results. Batches of listed keys were reliable. This is not a fault of the services, which are built mainly for interactive queries; bulk extraction is a different use. Therefore a client that extracts in bulk must check completeness, and ideally correctness, against the server's own counts. RDF Portal also offers its datasets as files, which may suit a full rebuild better than SPARQL.

A mine is built, not edited

InterMine serves objects from a stored copy and tracks which source set each value, so correcting one source in a mine that is already loaded proved fragile. For a pipeline that regenerates everything from RDF, the natural unit of update is a full rebuild. The build is not yet scripted from start to finish; we see that as the most important next step.

Future Work

- Script the whole build, from RDF to a running mine, including the trimmed web configuration, templates, search index and postprocessing, so that a mine can be rebuilt for each RDF Portal release.
- Extend the demonstration to a full HumanMine build: full-scale loads of the panel-limited sources, more of HumanMine's datasets, and the open mapping decisions.
- Load genome coordinates (FALDO), and GO annotations, which need an intermediate object that the current mapping cannot express.
- Add integration keys where HumanMine has none (GWASResult) or only a draft (Pathway, MeSH terms).
- Report the non-ASCII loading bug to InterMine; rdfc2im currently works around it.
- Propose additions to rdf-config that would help automated consumers, and validate the mapping files with SSSOM tools.
- Test the approach on another organism and another mine, and compare SPARQL extraction with RDF Portal's file downloads.

Acknowledgements

We would like to thank the Database Division of Life Sciences and the National Institute of Genetics for their support, and the organizers of the 2026 DBCLS BioHackathon in particular. R.Y.N. was supported by JST DICP Grant Number JPMJND2206, GteX Grant Number JPMJGX23B2, and JSPS KAKENHI Grant Numbers 23KK0122, 24K01889, and 25K09119 to Motomu Matsui at the Institute for Chemical Research, Kyoto University. The authors benefited from input, perspectives, support and encouragement from many other participants of the 2026 DBCLS BioHackathon. Claude (Anthropic) was used as programming and writing assistant during this project.

References

Database Center for Life Science. (2026a). RDF Portal [Accessed 18 September 2026] [citesAs-DataSource].

- Database Center for Life Science. (2026b). rdf-config: Describe RDF models and generate SPARQL queries, schema charts and more [Accessed 18 September 2026] **[cito:usesDataFrom]**.
- Kalderimis, A., Lyne, R., Butano, D., Contrino, S., Lyne, M., Heimbach, J., Hu, F., Smith, R., Štěpán, R., Sullivan, J., & Micklem, G. (2014). InterMine: Extensive web services for modern biology. *Nucleic Acids Research*, 42(W1), W468–W472. <https://doi.org/10.1093/nar/gku301> **[cito:citesAsAuthority]**.
- Kawashima, S., Katayama, T., Hatanaka, H., Kushida, T., & Takagi, T. (2018). NBDC RDF portal: A comprehensive repository for semantic data in life sciences. *Database*, 2018, bay123. <https://doi.org/10.1093/database/bay123> **[cito:citesAsAuthority]**.
- Lyne, R., Bazaga, A., Butano, D., Contrino, S., Heimbach, J., Hu, F., Kalderimis, A., Lyne, M., Reierskog, K., Stepan, R., Sullivan, J., Wise, A., Yehudi, Y., & Micklem, G. (2022). HumanMine: Advanced data searching, analysis and cross-species comparison. *Database*, 2022, baac054. <https://doi.org/10.1093/database/baac054> **[cito:citesAsAuthority]**.
- Matentzoglou, N., Balhoff, J. P., Bello, S. M., Bizon, C., Brush, M., Callahan, T. J., Chute, C. G., Duncan, W. D., Evelo, C. T., Gabriel, D., Graybeal, J., Gray, A., Gyori, B. M., Haendel, M., Harmse, H., Harris, N. L., Harrow, I., Hegde, H. B., Hoyt, A. L., ... Mungall, C. J. (2022). A simple standard for sharing ontological mappings (SSSOM). *Database*, 2022, baac035. <https://doi.org/10.1093/database/baac035> **[cito:usesMethodIn]**.
- Smith, R. N., Aleksic, J., Butano, D., Carr, A., Contrino, S., Hu, F., Lyne, M., Lyne, R., Kalderimis, A., Rutherford, K., Stepan, R., Sullivan, J., Wakeling, M., Watkins, X., & Micklem, G. (2012). InterMine: A flexible data warehouse system for the integration and analysis of heterogeneous biological data. *Bioinformatics*, 28(23), 3163–3165. <https://doi.org/10.1093/bioinformatics/bts577> **[cito:citesAsAuthority]**.